

Universidad Nacional del Centro de la
Provincia de Buenos Aires

FACULTAD DE CIENCIAS EXACTAS

Ingeniería de Sistemas



Trabajo Práctico de Aprobación del Final

Introducción a la Computación Evolutiva

Profesor a cargo: Yannibelli, Virginia

Buralli, Agustín: aburalli@alumnos.exa.unicen.edu.ar

Todesco, Sofia Anabel: stodesco@alumnos.exa.unicen.edu.ar

Introducción

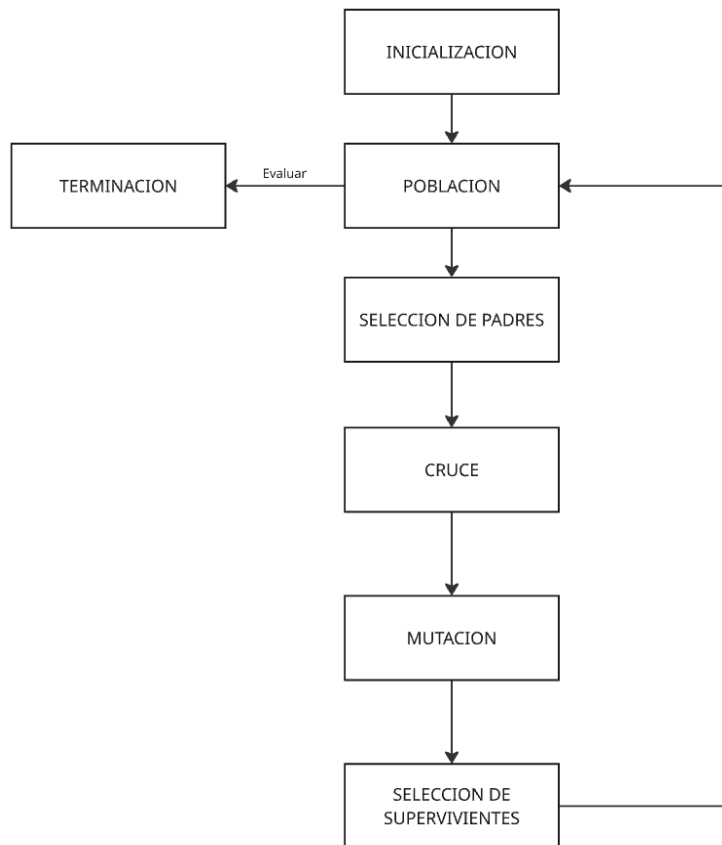
Este trabajo fue realizado con el objetivo de analizar, realizar y elegir alternativas y configuraciones de algoritmos evolutivos para la resolución heurística del problema del viajante (TSP).

Descripción del Diseño

La arquitectura principal es un pipeline, los módulos selección de padres, cruce, mutación, selección de supervivientes fueron diseñados previniendo su intercambio por otras variantes en tiempo de ejecución. Esto quiere decir que no importa cuantas variantes del algoritmo de selección de padres existan, que todas las implementaciones responden a la misma interfaz, lo mismo con los demás módulos. El módulo de inicialización es el único que no varía, posee parámetros configurables como tamaño de población y número de individuos generados por un método greedy, para evaluar las consecuencias y beneficios de tener un porcentaje de la población inicial “buena”.

Todas las interfaces son capaces de devolver una descripción textual de sus parámetros y su variante de implementación. Esto con el objetivo de facilitar el registro de cada experimento en el archivo de salida.

Una incógnita que surgió fue el cómo y dónde se realizara la visualización de los resultados obtenidos para su análisis. Java no es particularmente conocido por sus herramientas de visualización, por lo que decidimos importar el archivo de salida a un script de python que utiliza matplotlib para generar gráficos fácilmente.



Descripción de la Implementación

La primera etapa del proceso consiste en levantar el archivo .atsp que contiene la matriz con la información de los costos de los arcos y pasarla a una matriz de java para su fácil manejo. La clase ATSPReader se encarga de esto.

Como se explicó en el diseño, los métodos que se ubican en un mismo módulo del pipe implementan la misma interfaz.

En todas las etapas las muestras son representadas por un clase que representa la composición entre el tour y el fitness del mismo, se tomó esta decisión para poder asociar fácilmente el fitness de un tour para su utilización en métodos de ordenamiento en algoritmos como el de la ruleta o en comparación como el algoritmo de torneo. Dicha clase Sample tiene implementado la clase Comparable.

Para la instanciación de variantes de TSP se optó por utilizar un patrón builder, que es utilizado además de para conseguir configuraciones puntuales, para la generación de

configuraciones aleatorias usando la clase TSPRandomizer, que posee parámetros para aleatorizar la utilización de las variantes de las interfaces y la selección de parámetros random dentro de un rango factible. De esta manera se puede generar gran cantidad de instancias de tsp aleatorias para su posterior visualización.

LogWriter se encarga de la limpieza y creación del archivo output.txt y la clase Register de escribir en ella pasándole como parámetro el tsp. Al cual se le pueden pedir sus componentes y a su vez a sus componentes pedirles su descripción. Ya que todos implementan la interfaz IPrintable, que asegura que todas las clases tenga un método getStringValue(). Se podría haber hecho sobreescribiendo el toString también.

Una salida output.txt se veria asi:

Caso individual:

```
Initial Population: 50 Greedy initial solutions number: 5
Iterations number: 2000
Mating Pool Operator: Roulette Factor S: 1.85
CrossOver Operator: PMX Chance: 0.86
Mutation Operator: Inversion Chance: 0.056
Survivor Selection: ElitismX
[5684, 5684, 5684, 5684, 5684, 5678, 5678, 5678, 5678, 5678,
Best Composition: [4, 0, 35, 36, 1, 2, 3, 27, 28, 29, 5, 6,
Best Fitness: 5628
```

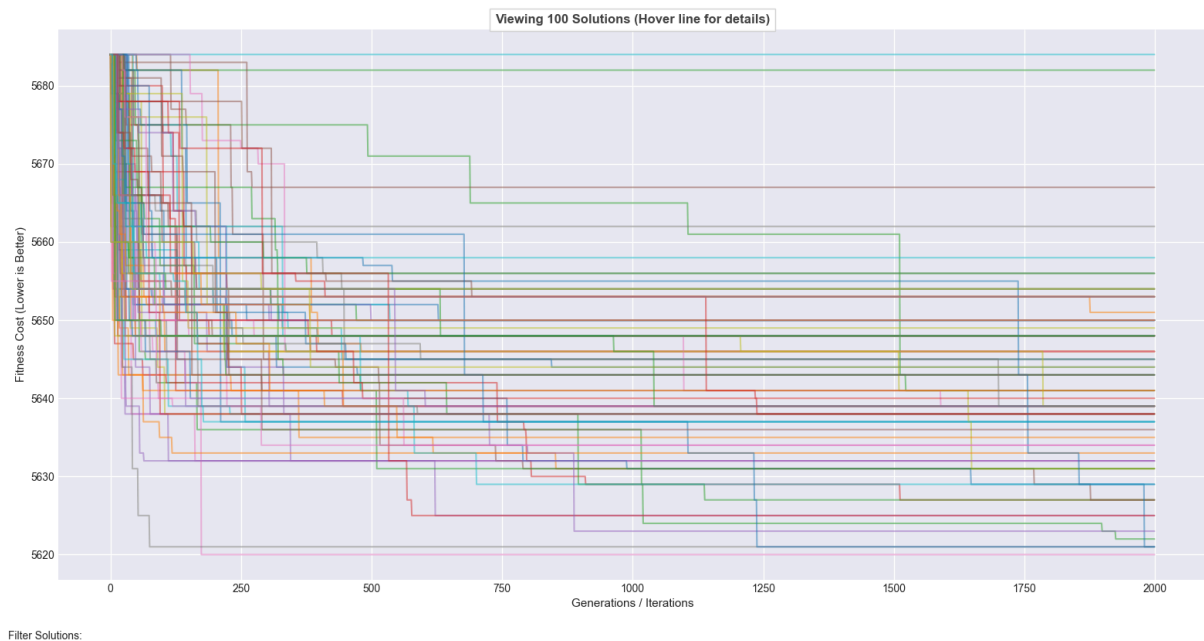
Luego de n iteraciones con una configuración específica:

```
Group: 3
Best fitness: 5620.0
Worst fitness: 5648.0
Average fitness: 5632.52
Standard Deviation: 7.547820877577848
Average execution time(Ms): 65.88 ms
```

Descripción de los Experimentos Computacionales Desarrollados

En primera instancia, realizamos 100 iteraciones del randomizador para que produzca 100 configuraciones aleatorias, y cada corrida del algoritmo realiza 2000 iteraciones. Se repitió este proceso un par de veces y de cada uno se fue anotando las configuraciones de las mejores soluciones y algunas peores para analizarlas más a fondo. Se realizó el registro en el siguiente archivo de hojas de cálculo: [TPF ICE Calculos](#). Las filas subrayadas en rojo dejaron de ser válidas debido a que hubo que realizar correcciones en el código.

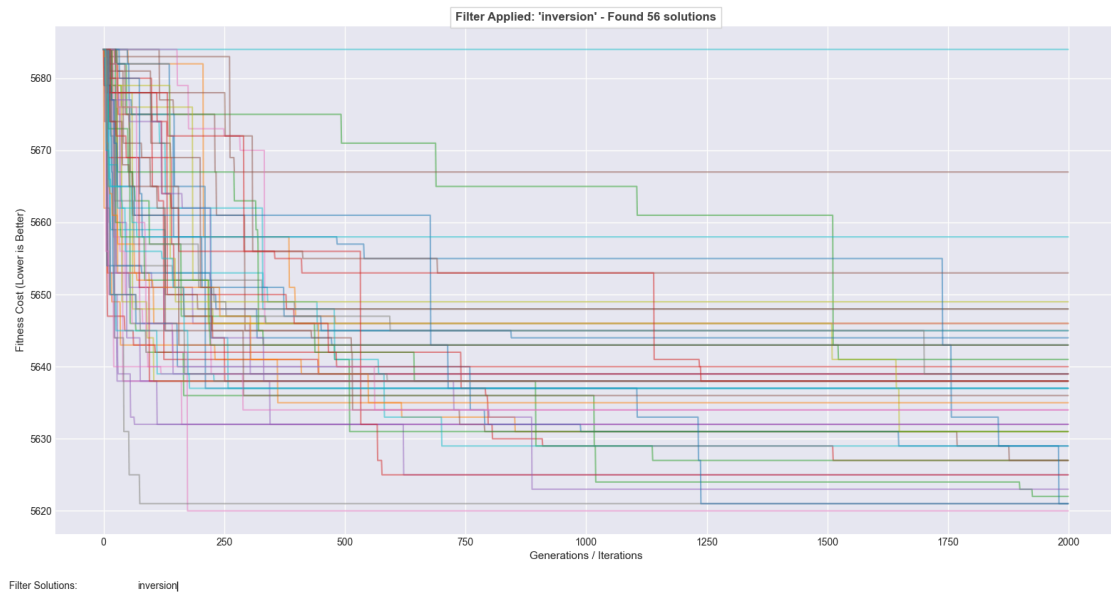
Ejemplo del resultado de un proceso:



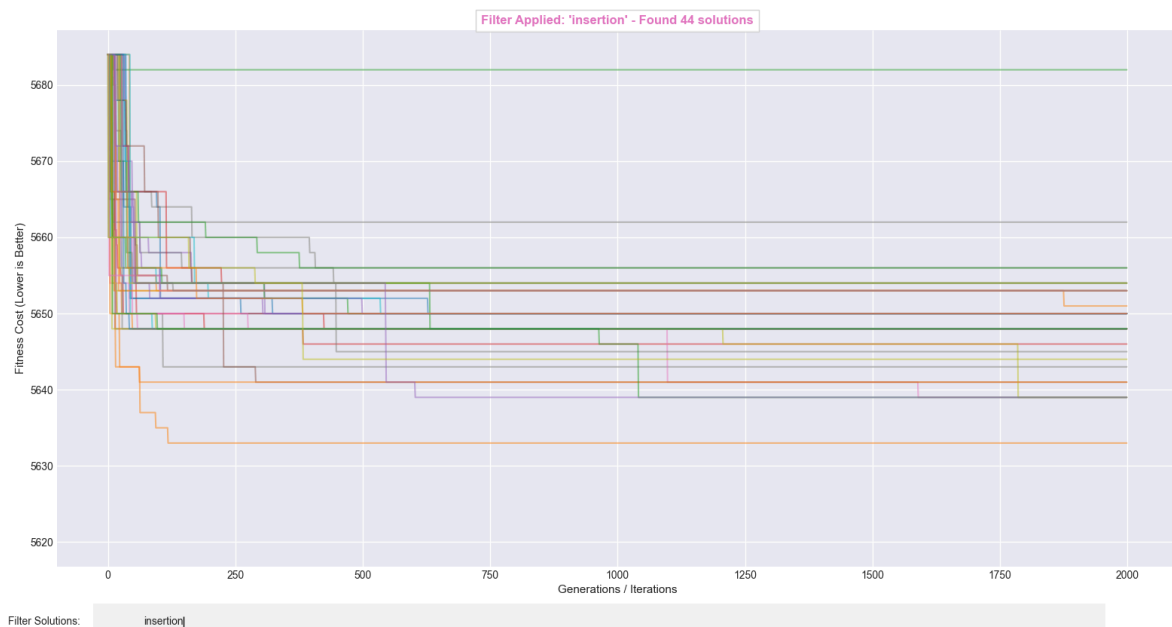
Luego con las 4 “mejores configuraciones” encontradas se realizaron 25 iteraciones con cada una y se las comparó. Se utilizaron las que en el archivo están subrayadas con amarillo.

Análisis Comparativo de Resultados

Algo interesante de resaltar, es que la mayoría de las mejores soluciones recolectadas usaron el operador de mutación inversión. Y esto es lo que se ve en el gráfico de cada operador:



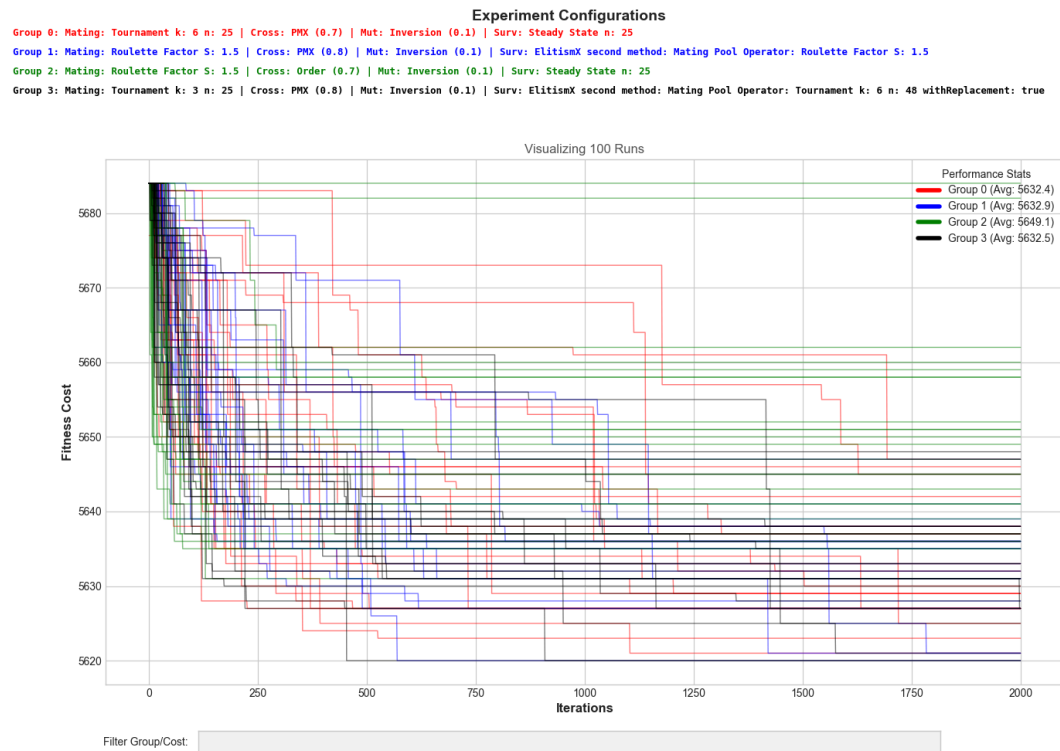
Inversión



Inserción

Como se observa, en el gráfico de operador de inversión hay resultados muy variados, están tanto los mejores como los peores en el. Y por otro lado, la inserción se mantiene más en el centro de los valores del fitness. Sus soluciones no forman parte de las mejores soluciones halladas.

Luego, al realizar la comparación de las 4 mejores configuraciones este fue uno de los resultados:



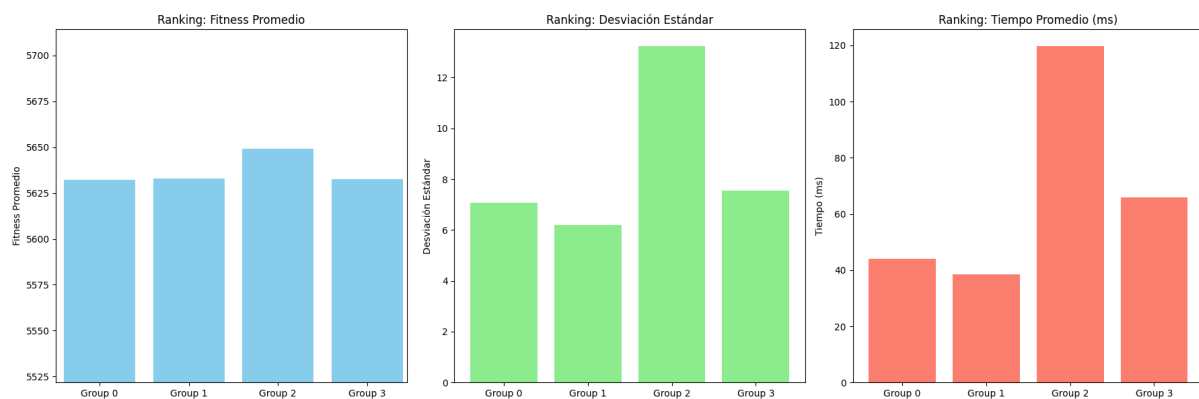
Las configuraciones son las siguientes:

Grupo	Color	Selección de Padres	Parámetros	Cruce (prob)	Mutación (prob)	Selección de Sobrevivientes	Parámetros
0	Rojo	Torneo	k: 6 reempl.: false	PMX (0.7)	Inversión (0.1)	Steady State	
1	Azul	Ruleta	s: 1.5	PMX (0.8)	Inversión (0.1)	Elitismo + Ruleta	s: 1.5
2	Verde	Ruleta	s: 1.5	Order (0.7)	Inversión (0.1)	Steady State	
3	Negro	Torneo	k: 3 reempl.: false	PMX (0.8)	Inversión (0.1)	Elitismo + Torneo	k: 6 reempl.: true

Se observó que en promedio la configuración verde (grupo 2) es la peor de las 4. Una parte de la mala performance de esta configuración probablemente sea que utiliza cruce por Orden. Como se predijo en el informe anterior, el cruce por PMX beneficia en el caso del problema del TSP donde las adyacencias importan.

Respecto a los parámetros, en el caso de la ruleta se observó que un α intermedio y equilibrado entre 1 y 2 da buenos resultados, ya que otorga una presión selectiva intermedia, ni mucha ni poca. En el caso del parámetro k en la selección por torneo, no se evidencia un cambio evidente en la performance de los algoritmos. Se sospecha que se debe a la previa selección de valores random sensatos entre 2 y 6.

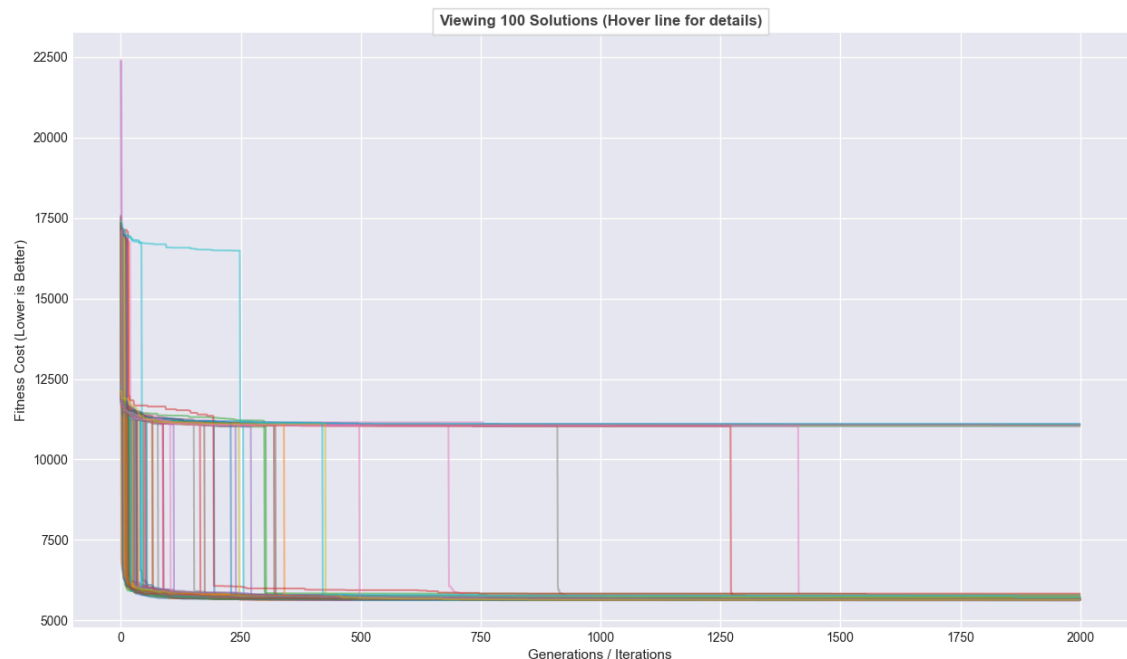
Si analizamos cada grupo homogéneamente:



En cada grupo, los fitness promedio son bastante similares. La configuración que más resalta es la del grupo 2, tiene el peor fitness promedio y el más alto desvío y tiempo promedios. Es curioso que siendo que es la que se ejecutó por más tiempo obtuvo el peor resultado. Se podría decir que, el hecho de que tome más tiempo no significa que va a encontrar el mejor valor.

Otra observación interesante, fue cuando se probó una ejecución quitando las 5 soluciones que se introdujeron con un método greedy. De igual manera, las soluciones finales son muy cercanas a las obtenidas con las soluciones greedy. Pero como se observa, el comienzo es mucho más difícil, comienzan en números muy altos, rápidamente se reduce, algunas soluciones en este caso se quedan alrededor de los 10000 de fitness y la mayoría

llega a los valores entre 5000 y 6000. Por lo tanto, podría decirse que la incorporación de soluciones greedy es útil principalmente para comenzar con una buena base.



Conclusiones

Para finalizar este informe, las conclusiones a las que llegamos mediante la experimentación son:

En primer lugar, la generación de la población inicial mostró mejores resultados al generar una pequeña porción de soluciones greedy al inicio. En caso contrario varias soluciones se quedan estancadas en valores muy mediocres.

En cuanto a la selección de padres, los mejores resultados surgieron utilizando Torneo o Ruleta con un valor del parámetro s lo más cercano al centro (1.5). En general cuando el valor se acercaba al 1, los resultados no eran muy buenos.

En cuanto al operador de cruce. Se puede observar que para este caso, el problema del viajante, en donde importan las adyacencias, PMX da mejores resultados que Orden.

En cuanto al operador de mutación, se podría decir que el más adecuado es el de Inversión. Inserción no llevó a buenos resultados.

Finalmente, en cuanto a la selección de sobrevivientes se considera que tanto Steady State como Elitismo + Ruleta o Torneo dan buenos resultados. Quizás un poco mejores usando Elitismo.